MGC-FINTECH

Full-Stack Financial and Business Management Web Application

1. Project Overview

MGC-FINTECH is a web-based financial and business management application designed to help individuals and companies manage financial transactions, customers, sales, invoices, inventory and business reports. The application provides a centralized dashboard where users can record business activities, monitor financial performance, manage stock, track customer payments and generate useful financial information. The project was developed using modern frontend and backend technologies, with cloud database services and online deployment.

---

2. Frontend Development

The frontend is the part of the application that users see and interact with.

Frontend programming languages

1. **TypeScript** – Used to develop the application's interactive functionality, data structures, components and application logic.
2. **HTML** – Provides the structural foundation for the web application.
3. **CSS** – Used to style the application, including layouts, colours, typography, responsive design and themes.

Frontend frameworks, libraries and tools

Technology	Practical use
React	Building reusable user interface components and managing application interactions.
Vite	Development server and frontend build tool.
Recharts	Displaying financial and business information through charts and graphs.
CSS	Responsive layouts, dark/light themes and visual styling.
TypeScript tooling	Type checking and maintaining more reliable application code.

Frontend features developed

- Interactive financial dashboard.
- Transaction entry and management.
- Customer registration and management.
- Sales, invoices and payment tracking.
- Inventory and stock management.
- Financial charts and reports.
- Multiple currency support.
- Language selection: English, French, German and Portuguese.
- Light and dark display themes.
- Data import and export capabilities.
- Responsive layouts for desktop, tablet and mobile screens.
- User profile editing.

---

3. Backend Development

The backend is responsible for processing application data, managing users, storing records and enforcing data security.

Backend languages

1. **SQL** – Used to define and manage database structures, queries, relationships and database operations.
2. **PL/pgSQL** – Used for PostgreSQL database functions and procedural business operations.

Backend technologies and tools

Technology	Practical use
Supabase	Backend-as-a-Service platform providing authentication, database and application data services.
PostgreSQL	Relational database used to store business and financial records.
Supabase Auth	User registration, login and authentication management.
Row Level Security (RLS)	Protecting user records so authenticated users can access only their permitted data.
Supabase Database Functions	Implementing important business operations, including sales processing, payments, inventory purchases and data clearing.
Supabase API	Connecting the React frontend to the backend database and authentication services.

Backend functionality developed

- User authentication and account management.
- Secure storage of financial and business records.
- Customer and product database management.
- Sales and invoice processing.
- Recording customer payments and outstanding balances.
- Inventory purchases and stock movement tracking.
- Automatic stock updates when sales and purchases are recorded.
- Financial transaction management.
- Account-specific data clearing and complete business data reset.
- Database access policies and relationships.

---

4. Development, Version Control and Deployment Tools

In addition to frontend and backend technologies, the following tools were used:

Tool	Purpose
Visual Studio Code / code editor	Editing and managing project source code.
Node.js	Providing the JavaScript runtime required for the development environment and build tools.
npm	Installing and managing project dependencies.
Git	Tracking source code changes and managing versions.
GitHub	Hosting the project repository and maintaining source code history.
Netlify	Hosting and deploying the web application online.
Chrome Developer Tools	Testing the interface, debugging and checking mobile responsiveness.

---

5. Practical Working Process: How MGC-FINTECH Was Built

Phase 1 – Requirements gathering and planning

I started by identifying the business problems the application should solve. The main requirements included financial transaction management, customer records, sales, inventory, financial reporting and secure data storage. I then organized these requirements into application modules.

Phase 2 – Frontend development

I used React and TypeScript to build the application's user interface and interactive components. I developed the dashboard and the main application sections, including Transactions, Customers, Sales, Inventory, Reports and Settings. CSS was used to create the visual design, themes and responsive layouts.

Phase 3 – Backend and database development

I configured Supabase and PostgreSQL as the backend infrastructure. I designed the database tables and relationships for customers, products, sales, sale items, payments, purchases, stock movements and financial transactions. I also implemented authentication, database functions and Row Level Security policies.

Phase 4 – Frontend and backend integration

I connected the React frontend to Supabase so that information entered by users could be securely processed and stored. For example, when a user creates a sale, the application links the sale to a customer, calculates the invoice total, records payments, updates inventory and generates the corresponding financial transaction. This integration allows different business modules to work together rather than operate as separate systems.

Phase 5 – Testing and troubleshooting

I tested the application by creating sample customers and products, processing sales, recording payments and checking inventory updates. I also tested database permissions, authentication, data-clearing functions and persistence of saved information. I used the browser developer tools and local development environment to identify and resolve issues.

Phase 6 – Version control and deployment

I used Git to track project changes and GitHub to maintain the source code repository. I connected the GitHub repository to Netlify and configured the production build using Vite. The application was successfully published at: `https://mgc-fintech.netlify.app/`

Phase 7 – Improvements and maintenance

I continued improving the application by introducing profile editing, mobile-responsive layouts and other usability enhancements. I tested profile information persistence and mobile layouts locally. The latest profile and mobile improvements were committed and pushed to GitHub, with their production deployment awaiting the next available Netlify deployment credits.

---

6. Example of an Integrated Business Workflow

One of the important achievements of MGC-FINTECH is the integration between its business modules. A typical workflow is:

1. Register a customer.
2. Add products to inventory.
3. Create a sales invoice for the customer.
4. Calculate the invoice total automatically.
5. Record the customer's payment.
6. Calculate the outstanding balance.
7. Update the available stock.
8. Record the financial transaction.
9. Display the updated information in the dashboard and reports.

This demonstrates how frontend components, backend processing and database relationships work together to support real business operations.

7. Technical Skills Demonstrated

Through the development of MGC-FINTECH, I gained practical experience in:

- Frontend development using React and TypeScript.
- Web styling and responsive design using CSS.
- Relational database design using PostgreSQL.
- SQL and database functions.
- Backend integration using Supabase.
- Authentication and database security.
- CRUD operations (Create, Read, Update and Delete).
- Financial and business workflow automation.
- Data validation and application testing.
- Version control using Git and GitHub.
- Cloud deployment using Netlify.
- Debugging and troubleshooting.

8. Conclusion

MGC-FINTECH demonstrates my practical understanding of full-stack web application development. The project gave me experience working across the frontend, backend, database, security, testing and deployment stages of a real application. My objective was not simply to build a website, but to develop an integrated business solution that can help users and companies manage financial and operational activities more efficiently.